جزوه جامع و بی‌نظیر فصل دوم (بخش ۷ و ۸: طراحی و ساختار سیستم‌عامل)

این جزوه بر اساس بخش‌های هفتم و هشتم از فصل دوم فایل ارسالی شما و با هدف پوشش **کامل، عمیق و مفهومی** تمام مطالب، تعاریف فنی، فرآیندها و مثال‌های ذکر شده تهیه شده است. هدف، درک کامل مفاهیم برای کسب بالاترین نمره است.

۷-۲: طراحی و پیاده‌سازی سیستم‌عامل

طراحی یک سیستم‌عامل جدید، یک چالش مهندسی عظیم و پیچیده است. در این بخش، با اصول و ملاحظاتی که در طراحی و پیاده‌سازی یک سیستم‌عامل مدرن نقش دارند، آشنا می‌شویم.

۲-۷-۱: اهداف طراحی *(Design Goals)*

اولین قدم، تعریف دقیق اهداف سیستم است. این اهداف به دو دسته اصلی تقسیم می‌شوند:

1. **اهداف کاربر (User Goals):** کاربران از یک سیستم‌عامل چه انتظاراتی دارند؟
   - **راحتی در استفاده و یادگیری:** سیستم باید بصری و کاربرپسند باشد.
   - **قابلیت اطمینان (Reliable):** سیستم باید پایدار باشد و به ندرت دچار خطا شود.
   - **امنیت (Secure):** داده‌ها و حریم خصوصی کاربر باید محفوظ بماند.
   - **سرعت (Fast):** سیستم باید پاسخ‌دهی سریعی داشته باشد.
1. **اهداف سیستم (System Goals):** توسعه‌دهندگان و مدیران سیستم چه انتظاراتی دارند؟
   - **سهولت** در طراحی، پیاده‌سازی **و نگهداری.**
   - **انعطاف‌پذیری (Flexible):** قابلیت افزودن ویژگی‌های جدید و تطبیق با سخت‌افزارهای مختلف.
   - **کارآمدی (Efficient):** استفاده بهینه از منابع سخت‌افزاری (CPU، حافظه و...).
   - **عاری از خطا بودن (Error-free).**

نکته کلیدی: بین این اهداف، اغلب **تضاد (Trade-off)** وجود دارد. برای مثال، یک سیستم بسیار کاربرپسند (مانند macOS) ممکن است به اندازه یک سیستم ساده‌تر (مانند یک نسخه از لینوکس) انعطاف‌پذیر نباشد. طراحان سیستم‌عامل باید بر اساس نوع سیستم (رومیزی، موبایل، سرور، بلادرنگ) بین این اهداف تعادل برقرار کنند.

۲-۷-۲: تفکیک سیاست از راهکار *(Policy vs. Mechanism)*

این یک اصل طراحی بسیار مهم در مهندسی نرم‌افزار و به ویژه در سیستم‌عامل است.

- **راهکار (Mechanism):** به **"چگونگی"** انجام یک کار اشاره دارد. راهکار، ابزارها و قابلیت‌های پایه را فراهم می‌کند.
  - **مثال:** قابلیت تنظیم اولویت برای پردازه‌ها یک راهکار است.
- **سیاست** (Policy): به **"چیستی"** کاری که باید انجام شود، اشاره دارد. سیاست، نحوه استفاده از راهکارها را برای رسیدن به یک هدف خاص مشخص می‌کند.
  - **مثال:** اینکه "پردازه‌های تعاملی (Interactive) باید اولویت بالاتری نسبت به پردازه‌های پس‌زمینه داشته باشند"، یک سیاست است.

چرا این تفکیک مهم است؟تفکیک سیاست از راهکار، باعث **انعطاف‌پذیری** سیستم می‌شود. سیاست‌ها ممکن است در طول زمان یا بر اساس نیاز کاربر تغییر کنند. اگر راهکار به صورت کلی و مستقل از سیاست طراحی شده باشد، می‌توان با تغییر یک پارامتر ساده، سیاست را عوض کرد، بدون اینکه نیازی به تغییر کد اصلی سیستم (راهکار) باشد.

- **مثال:**
  - **سیستم‌عامل‌های مبتنی بر ریزهسته (Microkernel):** این سیستم‌ها با فراهم کردن مجموعه‌ای از راهکارهای بسیار پایه‌ای و رها از سیاست، این اصل را به نهایت می‌رسانند.
  - **ویندوز و macOS:** این سیستم‌عامل‌ها برای ایجاد یک تجربه کاربری یکپارچه، سیاست و راهکار را به طور تنگاتنگی با هم ادغام کرده‌اند.
  - **لینوکس:** هسته لینوکس یک راهکار استاندارد (مثلاً برای زمان‌بندی) ارائه می‌دهد، اما به دلیل متن‌باز بودن، به راحتی می‌توان راهکار را تغییر داد یا اصلاح کرد تا از سیاست‌های متفاوتی پشتیبانی کند.

۲-۷-۳ : پیاده‌سازی *(Implementation)*

- **زبان برنامه‌نویسی:**
  - **در گذشته:** سیستم‌عامل‌های اولیه کاملاً به **زبان اسمبلی** نوشته می‌شدند تا حداکثر کارایی و کنترل روی سخت‌افزار را داشته باشند.
  - **امروزه:** اکثر سیستم‌عامل‌های مدرن با ترکیبی از زبان‌ها پیاده‌سازی می‌شوند:
    - بخش‌های حساس به کارایی و سطح پایین هسته (مانند راه‌انداز وقفه) ممکن است هنوز به **زبان اسمبلی** نوشته شوند.
    - بدنه اصلی هسته و درایورها معمولاً به زبان **C** نوشته می‌شوند.
    - کتابخانه‌های سیستمی و بخش‌های سطح بالاتر ممکن است به زبان‌های **C++** یا حتی زبان‌های مدرن‌تری مانند **جاوا** (در مورد اندروید) نوشته شوند.
- **مزایای استفاده از زبان‌های سطح بالا:**
  - **توسعه سریع‌تر و آسان‌تر.**
  - **درک و عیب‌یابی ساده‌تر کد.**
  - **قابلیت حمل (Portability):** انتقال سیستم‌عامل به یک معماری سخت‌افزاری جدید، با کامپایل مجدد کد، بسیار آسان‌تر می‌شود.
- **معایب:**
  - کاهش جزئی سرعت و افزایش مصرف حافظه. اما امروزه با پیشرفت کامپایلرها، این معایب تا حد زیادی کمرنگ شده‌اند و کامپایلرهای بهینه‌ساز اغلب می‌توانند کدی کارآمدتر از کد نوشته شده توسط انسان تولید کنند.

۲-۸ : ساختار سیستم‌عامل (Operating-System Structure)

یک سیستم‌عامل بزرگ باید به صورت **ماژولار** طراحی شود تا قابل مدیریت و توسعه باشد. در این بخش، با رویکردهای مختلف برای ساختاردهی به هسته سیستم‌عامل آشنا می‌شویم.

۲-۸-۱ : ساختار یکپارچه *(Monolithic Structure)*

- **ایده:** ساده‌ترین ساختار که در آن هیچ ساختار مشخصی وجود ندارد! تمام اجزای هسته (زمان‌بند CPU، مدیر حافظه، سیستم فایل و درایورها) در یک فایل باینری بزرگ و واحد کامپایل شده و همگی در **یک فضای آدرس واحد** در مد هسته اجرا می‌شوند.
- **مثال:** یونیکس اولیه و تا حد زیادی لینوکس.

- **مزیت:**
  - **کارایی بسیار بالا:** ارتباط بین اجزای مختلف هسته، صرفاً یک فراخوانی تابع ساده است و هیچ سربار اضافی (مانند عبور پیغام) وجود ندارد.
- **معایب:**
  - **پیچیدگی در توسعه و نگهداری:** تمام اجزا به شدت به هم وابسته‌اند (Tightly Coupled). یک خطا در یک درایور می‌تواند کل سیستم را از کار بیندازد.
  - **گسترش دشوار:** افزودن یک قابلیت جدید نیازمند کامپایل مجدد کل هسته است.

۲-۸-۲ : رویکرد لایه‌ای *(Layered Approach)*

- **ایده:** سیستم‌عامل به چندین **لایه (Layer)** تقسیم می‌شود که به صورت سلسله مراتبی روی هم قرار گرفته‌اند.
  - **لایه ۰:** سخت‌افزار.
  - **لایه N (بالاترین لایه):** رابط کاربر.
- **قانون اصلی:** هر لایه فقط مجاز است از توابع و سرویس‌های لایه **مستقیماً پایینی** خود استفاده کند.
- **مزیت:**
  - **سادگی در طراحی و عیب‌یابی:** هر لایه به صورت مستقل از لایه‌های بالایی قابل تست و عیب‌یابی است، زیرا فقط به لایه‌های پایینی که قبلاً تست شده‌اند، وابسته است. این رویکرد، انتزاع (Abstraction) را به خوبی پیاده‌سازی می‌کند.
- **معایب:**
  - **کارایی پایین:** یک درخواست کاربر ممکن است برای رسیدن به سخت‌افزار، مجبور باشد از چندین لایه عبور کند که هر کدام سربار خود را دارند.
  - **دشواری در تعریف لایه‌ها:** تعریف دقیق وظایف هر لایه به طوری که این قانون سخت‌گیرانه رعایت شود، بسیار دشوار است.

۲-۸-۳ : ریزهسته‌ها *(Microkernels)*

- **ایده:** تا جای ممکن هسته را کوچک کنیم! این رویکرد، تمام سرویس‌های غیرضروری را از هسته (که در مد هسته اجرا می‌شود) خارج کرده و آن‌ها را به صورت **برنامه‌های سطح کاربر (که سرور نامیده می‌شوند)** در فضاهای آدرس مجزا پیاده‌سازی می‌کند.
- **وظایف اصلی ریزهسته:**
  - مدیریت پردازه و حافظه (در سطح پایین).
  - فراهم کردن یک **مکانیزم ارتباطی** (معمولاً **عبور پیغام (Message Passing)**) بین برنامه‌های کاربر (کلاینت‌ها) و سرورهای ارائه‌دهنده سرویس.
- **فرآیند:** اگر یک برنامه کاربر بخواهد فایلی را بخواند، یک پیغام به **سرور سیستم فایل** ارسال می‌کند. این ارتباط به صورت غیرمستقیم و از طریق ریزهسته انجام می‌شود.
- **مزایا:**
  - **قابلیت اطمینان و امنیت بالا:** خرابی در یک سرور (مثلاً سرور فایل)، کل سیستم را از کار نمی‌اندازد.
  - **گسترش‌پذیری آسان:** افزودن سرویس‌های جدید به راحتی و با اضافه کردن یک سرور جدید در فضای کاربر انجام می‌شود.
  - **قابلیت حمل بالا.**
- **معایب:**
  - **کارایی پایین‌تر:** سربار ناشی از عبور پیغام بین کلاینت‌ها و سرورها (که نیازمند کپی کردن داده و تغییر زمینه بین پردازه‌هاست) می‌تواند قابل توجه باشد.

۲-۸-۴ : ماژول‌ها *(Modules)*

- **ایده:** این رویکرد، بهترین ویژگی‌های ساختار یکپارچه و ریزهسته را با هم ترکیب می‌کند.
- **عملکرد:** هسته یک مجموعه از اجزای اصلی و ضروری را در خود دارد، اما می‌تواند سرویس‌های دیگر (مانند درایورهای دستگاه یا سیستم‌های فایل جدید) را به صورت **ماژول‌های قابل بارگذاری (Loadable Kernel Modules - LKMs)** در زمان بوت یا در حین اجرای سیستم، به صورت **پویا** به خود اضافه یا از خود حذف کند.
- **مثال:** لینوکس به طور گسترده از این رویکرد استفاده می‌کند. وقتی شما یک فلش مموری را به سیستم متصل می‌کنید، هسته لینوکس ماژول مربوط به درایور USB و سیستم فایل آن را به صورت خودکار بارگذاری می‌کند.
- **مزایا:**
  - **انعطاف‌پذیری:** مانند ریزهسته، قابلیت گسترش پویا دارد.
  - **کارایی:** مانند ساختار یکپارچه، ارتباط بین ماژول‌ها مستقیم و از طریق فراخوانی تابع است و سربار عبور پیغام وجود ندارد.

۲-۸-۵ : سیستم‌های ترکیبی *(Hybrid Systems)*

در عمل، اکثر سیستم‌عامل‌های مدرن از یک ساختار خالص پیروی نمی‌کنند، بلکه ترکیبی از رویکردهای مختلف هستند تا به تعادلی بین کارایی، امنیت و انعطاف‌پذیری دست یابند.

- **macOS و iOS:**
  - این سیستم‌عامل‌ها ساختاری ترکیبی و لایه‌ای دارند.
  - هسته آن‌ها به نام **Darwin** خود یک سیستم ترکیبی است که شامل **ریزهسته Mach** (برای مدیریت سطح پایین پردازه، حافظه و IPC) و بخش‌هایی از **هسته BSD یونیکس** (برای ارائه سرویس‌های سطح بالاتر مانند API POSIX) می‌باشد.
  - برای حل مشکل کارایی ریزهسته، اپل تمام این اجزا (Mach, BSD, I/O Kit) را در یک فضای آدرس واحد (مد هسته) اجرا می‌کند.
- **اندروید:**
  - ساختاری لایه‌ای دارد که در پایین‌ترین سطح آن، **هسته لینوکس** (که برای نیازهای موبایل اصلاح شده) قرار دارد.
  - در لایه‌های بالاتر، کتابخانه‌های بومی (نوشته شده به C/C++)، **محیط زمان اجرای اندروید (ART)** و چارچوب‌های برنامه‌نویسی جاوا قرار دارند.
  - **لایه تجرید سخت‌افزار (Hardware Abstraction Layer - HAL):** یک لایه مهم